

Finite Impulse Response (FIR) Filter Design

64-tap 16-bit FIR Filter in Verilog

Junyi (Charlie) Mei
B.S. Computer Engineering
Columbia University
jm5912@columbia.edu

Abstract—This report elaborates the design, simulation, analysis, and application of my 64-tap 16-bit Finite Impulse Response (FIR) Filter. The entire design was constructed with Verilog as our Hardware Description Language and tested with ModelSim and Synopsys Design Compiler as our Computer-Aided Design(CAD) Tools. The simulation results were compared with the theoretical value generated with MATLAB. The chip design was verified by using Synopsys PrimeTime to indicate whether the circuit built was efficient and applicable. Finally, case study was included in the last section to demonstrate the real world application of our filter.

Index Terms—FIR, MATLAB, Verilog, ModelSim, Synopsys Design Compiler & PrimeTime, Finite Discrete Convolution, Signal Processing

I. INTRODUCTION

An FIR Filter is a filter whose impulse response is the summation of finite duration of the multiplication of given coefficient and current and previous input. FIR Filter is widely used in Digital Signal Processing(DSP) applications such as audio & image processing, communication system, and etc.

The specific FIR Filter designed was a 64-tap 16-bit FIR Filter. For a given input signal $x[n]$, the output signal $y[n]$ is:

$$y[n] = \sum_{i=0}^N b_i * x[n-i]$$

Where b_i is the i^{th} element(0-indexed) of coefficient array b . The 64-tap attribute was achieved by picking $N = 63$, thus the formula became:

$$y[n] = \sum_{i=0}^{63} b_i * x[n-i]$$

The 16-bit attribute was represented by having input and output signal as 16-bit binary inputs. Specifically, input signal and coefficient, $x[n]$ and b_i , were 2's complement fixed point with first 4-bit as integer part and the rest 12-bit as fraction part. And output signal, $y[n]$, was 16-bit half precision floating point (IEEE 754 standard).

Since the coefficient array b is finite(64), the output signal was essentially the computation of finite discrete convolution

of coefficient and input signals. As elaborated in Section II(Signal Processing Basics), convolution was used as an important tool in time-domain to filter out specific range in frequency-domain.

In this report, we will go over fundamental principles in signal processing to understand the motivation of our design. While this report mainly focused on the simulation in Verilog and Synopsys Design Compiler and PrimeTime, MATLAB was used to generate input signals and compute the theoretical output signals before applying them in Verilog Simulation. The output signals of Verilog simulation were compared to the theoretical output signals from MATLAB to validate the effectiveness of the design.

Specifically, the input signals were fed through the FIR Filter at the rate of 10kHz. The FIR Filter Core (FIR Core), which computes the output signal, had a clock frequency of 1MHz. To ensure the correct output can be obtained, the faster clock (FIR Core) ran 100 times faster than the slower clock (FIR Filter). The above clock frequency was used for the Verilog module design and synthesis.

II. SIGNAL PROCESSING BASICS

Essentially, FIR Filter takes in finite discrete convolution of two signals and, if implemented correctly, can be used to filter out certain frequency of the signals. To understand the application of FIR Filter, I will go over some fundamental concepts in signal processing (well this is not a Signals & Systems course).

A. Unit Step Function

Unit-step function, denoted as $u(t)$, is defined as:

$$u(t) = \begin{cases} 1 & \text{for } t \geq 0 \\ 0 & \text{for } t < 0 \end{cases}$$

B. Unit Impulse Function

Unit-impulse function, denoted as $\delta(t)$, is defined as:

$$\delta(t) = 0 \text{ for } t \neq 0 \text{ and } \int_{-\infty}^{\infty} \delta(t)dt = 1$$

Unit-impulse function is an infinitely narrow and tall rectangle with area of 1 located at $t = 0$

C. Causal Signal

A signal $f(t)$ is causal if:

$$f(t) = 0 \text{ for } t < 0$$

Thus a causal signal $f(t)$ is equivalent to $f(t)u(t)$

D. Convolution

Given two signals $f(t)$ and $g(t)$, the convolution of the two signals $c(t)$ is:

$$f(t) * g(t) = \int f(\tau)g(t - \tau)d\tau$$

E. Convolution of Causal Signals

If both signals are causal, then the convolution of two such signals $f(t)$ and $g(t)$ is:

$$f(t) * g(t) = \int_0^t f(\tau)g(t - \tau)d\tau$$

If $f(t)$ is only non-zero from 0 to 64 (you will see why in a second), the convolution becomes:

$$f(t) * g(t) = \int_0^{64} f(\tau)g(t - \tau)d\tau$$

F. Finite Discrete Convolution

By definition, integral is the summation of continuous and infinitely small pieces of area under a curve. By taking its discrete form ($\Delta\tau$) the integral above can be written as:

$$\sum_{n=0}^{(n+1)\Delta\tau=64} f(n\Delta\tau)g(t - n\Delta\tau)$$

By taking $\Delta\tau = 1$, the finite discrete convolution above becomes:

$$\sum_{n=0}^{n=63} f(n)g(t - n)$$

The above form is exactly output signal computation of my FIR Filter. With $f(n)$ as coefficient b_i , $g(t - n)$ as previous input $x[n - i]$, and the summation as the output signal $y[n]$.

G. Zero Filter State Assumption

In section 1E, we have shown that the finite discrete convolution only holds with $t > 64$. This is because there is no negative time signal. To make reality matches with the theoretical calculation. We will assert zero filter state assumption by setting filter state to all zeros before any signals passed in. Therefore, for the first 64 signals, all of the signals will be used for calculation with rest of the zero filter states not affected the result.

III. FIR FILTER APPLICATION: LOW PASS FILTER

With the coefficient chosen properly, FIR Filter can filter out signals with certain frequency. To understand how such coefficients are determined, we will introduce Fourier Transform, a technique to convert signals between time-domain and frequency-domain.

A. Fourier Transform

The Fourier Transform of a signal $f(t)$, denoted as $F(w)$, and also written as $\mathcal{F}\{f(t)\}$ is:

$$F(w) = \int f(t)e^{-j\omega t}dt$$

Where j is imaginary unit with $j^2 = -1$.

$f(t)$ is also the inverse Fourier Transform of $F(w)$, also written as $\mathcal{F}^{-1}\{F(w)\}$:

$$f(t) = \frac{1}{2\pi} \int F(w)e^{j\omega t}dw$$

B. Fourier Transform of Useful Functions

The following Fourier Transform are given for calculation in later section. Though the steps were omitted, the answer was directly derived from basic calculus knowledge.

$$\mathcal{F}\{e^{j\omega_0 t}\} = 2\pi\delta(w - \omega_0)$$

C. Useful Properties

Given Fourier Transform of $f(t)$ as $F(w)$ and $g(t)$ as $G(w)$, we have:

$$\begin{aligned} \mathcal{F}\{kf(t) + mg(t)\} &= kF(w) + mG(w) \\ \mathcal{F}\{f(t) * g(t)\} &= F(w)G(w) \end{aligned}$$

The properties above are significant as we will see in the next section.

D. Time-domain vs. Frequency Domain

Essentially, $f(t)$ represents the signal in time-domain and its Fourier Transform, $F(w)$, represents the signals in frequency domain. To demonstrate this, we will use simple sinusoid $f(t) = \cos(2\pi 10t)$ as an example. For convenience, we will set $\omega_0 = 20\pi$ so that $f(t) = \cos(\omega_0 t)$.

Given $\cos(\omega_0 t) = \frac{1}{2}(e^{j\omega_0 t} + e^{-j\omega_0 t})$ and applying the results above, the Fourier Transform $F(w)$ is:

$$F(w) = 2\pi[\delta(w - \omega_0) + \delta(w + \omega_0)]$$

E. Properties

Given the Fourier Transform of $f(t)$ as $F(w)$ and $g(t)$ as $G(w)$, the Fourier Transform of convolution of two signals in time-domain is the multiplication of two signals in frequency-domain:

$$f(t) * g(t) \leftrightarrow F(w)G(w)$$

IV. MATLAB

The implementation of the FIR Filter was first simulated with MATLAB code. While there are multiple test files generated to verify the functionality of each block of the FIR Filter (ALU, IMEM, CMEM, FIFO), I will discuss specifically with the test file `test_fir.m` which generate the input data, coefficient, and expected output to verify the FIR Filter design. The file `test_fir.m` generated the following three files:

A. fir_input.txt

This file contains 16384 16-bit binary fixed point input signal. To prevent overflow during computation, the data is generated by choosing random number from -1 to 1 and then converted into 16-bit binary fixed point.

B. coef.txt

This file contains 64 16-bit binary fixed point coefficient that will be used to compute output signal for each input signal. To prevent overflow during computation, the data is generated by choosing random number from -1 to 1 and then converted into 16-bit binary fixed point.

C. fp_output.txt

This file contains 16384 16-bit half precision binary floating point output signal obtained with the given input data and coefficient. This file will be used in the Verilog testbench to verify the functionality of my FIR Filter.

D. Zero Filter State Assumption

For the testing purpose, the system is considered to start at $t = 0$. Therefore there is no instruction memory before the first input data comes in. Therefore, the testing will start with the filter state (IMEM in FIR module) reset with all registers to be 0.

V. DESIGN DIAGRAMS

A. Control ASM Chart

The Algorithmic State Machine (ASM) is a useful tool for development of datapath and controller, since it details the procedure of the program. As shown in Figure 1 below, the ASM Chart describes the sequential flow of the program which determine the state transitions in between 5 states.

B. Datapath Block Diagram

Given the ASM Chart above, we can put them together to build the datapath of FIR Filter. To show the design process, which will be helpful in the discussion of simulation in the next section, the block diagram for each component will be discussed before assembling them together into the final datapath.

1) **ALU**: The Arithmetic Logic Unit (ALU) for the FIR Filter is the core computation unit of the FIR Filter where it takes the multiplication of 2 16-bit 2's complement binary fixed point (din1, din2) and increment the current result. The output is a 16-bit half precision floating point (dout).

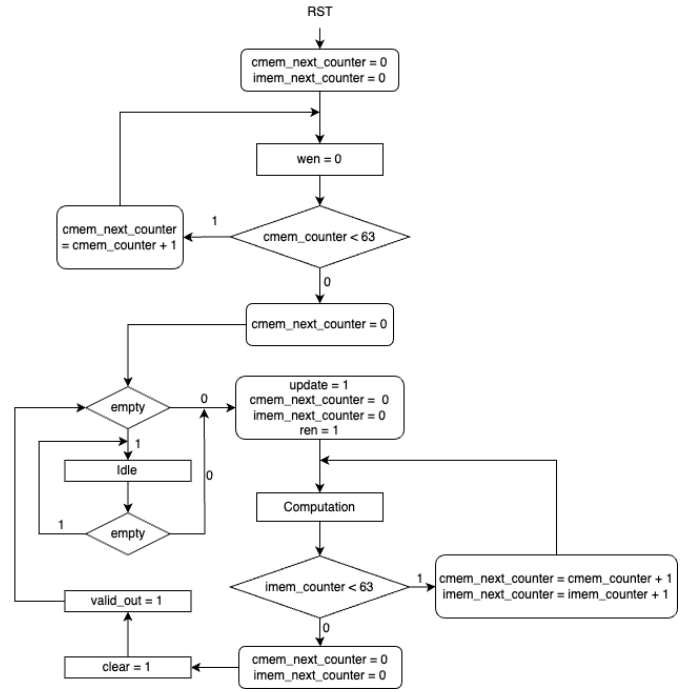


Fig. 1. ASM Chart for FIR Filter

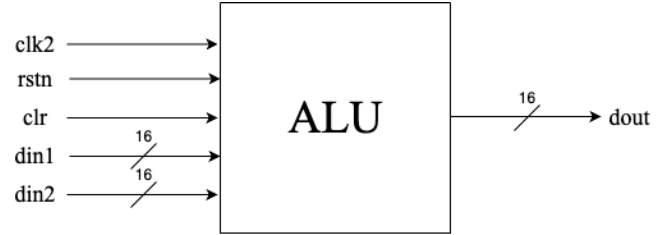


Fig. 2. ALU Block Diagram

2) **CMEM**: The Coefficient Memory (CMEM) is 64 16-bit memory block that can read and write coefficients into the memory block. This is automatically generated with the memory compiler by setting the specifics with the dimensions above and clock frequency to be 1MHz, which is the faster clock clk2's frequency. This is an active low system so data will be written into CMEM[a] with $wen = 1$ and read from CMEM[caddr] when $wen = 0$.

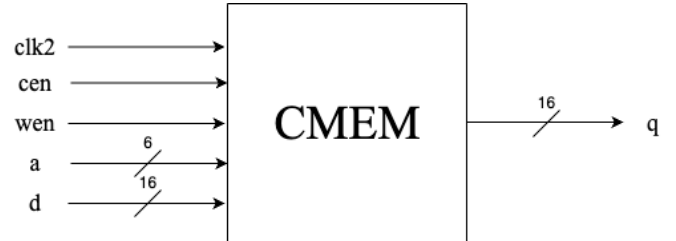


Fig. 3. CMEM Block Diagram

3) *IMEM*: The Instruction Memory (CMEM) is 64 16-bit memory block that contain the most recent 64 input signals. When *update* = 1, the IMEM block will act as a shift register with input data written into the front of the block with the original data shifted to the right by one position. When *update* = 0, data will be read from the IMEM[iaddr].

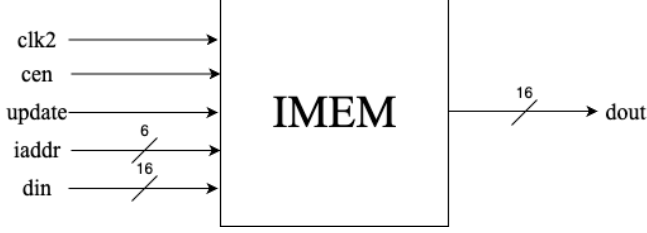


Fig. 4. IMEM Block Diagram

4) *FIFO*: The dual-clock FIFO (FIFO) is 1 16-bit memory block that captures input signal at the rate of slower clock frequency and read data at the rate of faster clock frequency. Data will be written when FIFO is empty and read when FIFO is not empty. The data being read will be fed into IMEM block when it is ready. The input signal *wen* (write enable) and *ren* (read enable) controls the mode of the FIFO operation (read or write)

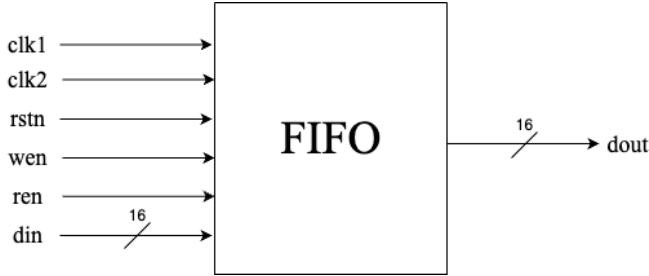


Fig. 5. FIFO Block Diagram

5) *Datapath Diagram*: Putting everything together by adding registers, counter, and multiplexer. The final datapath block diagram will look figure 6 below.

VI. SIMULATION

With the RTL Verilog code for FIR Core (fir.v), the gate-level netlist (fir.nl.v) is generated by Design Compiler. The simulation was conducted with the gate-level netlist, including tests that verify multiple functions of the FIR Filter. The following section will discuss how the test is constructed with the corresponding output.

A. Testbench

The testbench under qsim_dc used random inputs generated in MATLAB. We will use the 64 coefficient in coef.txt file and the first 512 out of 16384 inputs in fir_input.txt to test the coefficients can be loaded and the output signals can be computed correctly. The first 512 out of 16384 outputs in

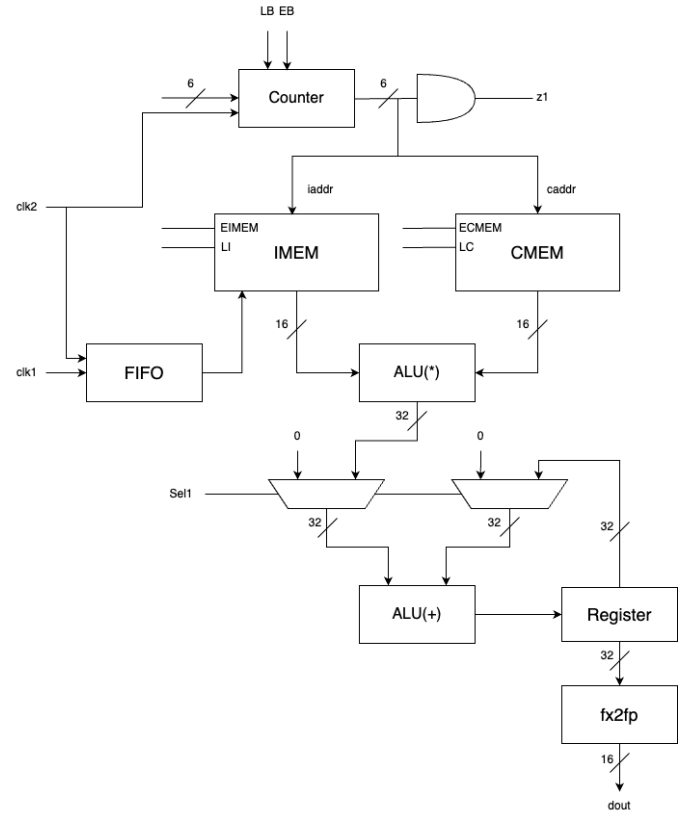


Fig. 6. Datapath Block Diagram

fp_output.txt were used to compare the output signals with the expected output.

B. CMEM Loading

Before receiving input signals, the 64 16-bit coefficients were loaded into the CMEM block. From figure 11, we can see that after reset was released at the rising edge of slower clock (clk1), cin was fed with input from coef.txt with corresponding

C. IMEM Updating

IMEM represents the filter states, which is updated whenever there is new input signal fetched by FIFO block. During computation stage, IMEM was first updated with the new input signal and then was read by address iterately from 0 to 63. Figure 6 demonstrate that IMEM was successfully updated since there were one more non-zero input to be read before the signal becomes zero. Figure 7 also showed that IMEM can be updated when it is full since the output at address 0 is different between the two cycles.

D. Output signal

The output signal is only compared with the expected output when valid_out = 1. Figure 8 below shows one of the output signal which matches the expected output.

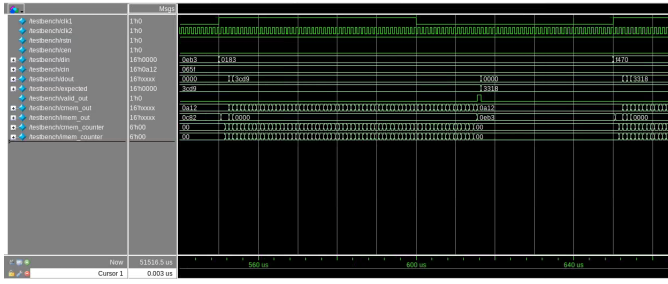


Fig. 7. IMEM updating when it is not full

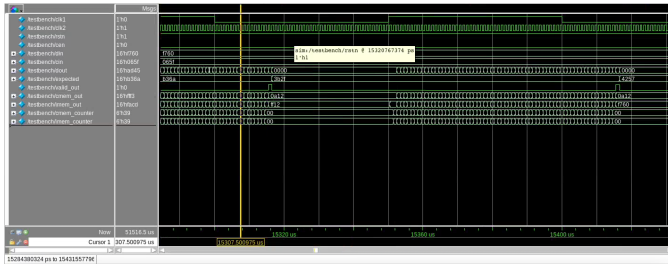


Fig. 8. IMEM updating when it is full

E. Simulation Result

The entire simulation for 512 input signals ran 51516500 nanoseconds with all output signals matched the expected outputs. Figure 9 below is a screenshot of the output message after running QuestaSim.

VII. METRICS

A. Area

As shown in figure 10, we obtained the following area constraints for building the physical module:

Combinational Area: 38905.92um²
Buffer and Inverter Area: 10022.40um²
Noncombinational Area: 36573.12um²
Macro and Black Box Area: 12671.39um²

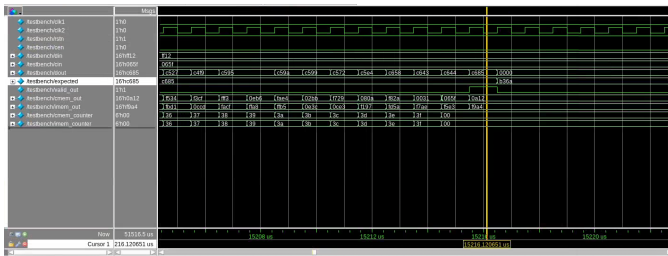


Fig. 9. Computed Output Signal vs. Expected Output Signal

```
# All tests passed
# ** Note: $finish : test_fir.v(173)
# Time: 51516500 ns Iteration: 1 Instance: /testbench
```

Fig. 10. qsim_dc Testbench Simulation Result

Putting all of the metrics above together, we obtain the total cell area to be:

Total Cell Area: 88150.43um²

```
3 *****
4 Report : area
5 Design : fir
6 Version: U-2022.12-SP7
7 Date : Sun Dec 15 04:15:58 2024
8 *****
9
10 Library(s) Used:
11
12 scx3_cmos8rf_lpvtt_tt_1p2v_25c (File: /courses/ee6321/share/ibm1
13 USERLIB (File: /homes/user/stud/fall24/jm5912/fir-project/mem_c
14
15 Number of ports: 139
16 Number of nets: 6217
17 Number of cells: 5976
18 Number of combinational cells: 4854
19 Number of sequential cells: 1120
20 Number of macros/black boxes: 1
21 Number of buf/inv: 2015
22 Number of references: 48
23
24 Combinational area: 38905.920739
25 Buf/Inv area: 10022.400398
26 Noncombinational area: 36573.118851
27 Macro/Black Box area: 12671.385742
28 Net Interconnect area: undefined (No wire load specified)
29
30 Total cell area: 88150.425332
31 Total area: undefined
```

Fig. 11. Area Report from Design Compiler

B. Timing and Max Clock Frequency

The maximum clock frequency of the FIR Filter is determined by the critical path delay of the circuit. Figure 11 and figure 12 demonstrate the minimum path delay and critical path delay as following:

Minimum Path Delay: 0.1490ns
Critical Path Delay: 30.6896ns

276	Startpoint: rstn (input port clocked by clk2)		
277	Endpoint: controller_0_imem_counter_reg_4_		
278	(removal check against rising-edge clock clk2)		
279	Path Group: **asyncl_default**		
280	Path Type: min		
281			
282			
283			
284	clock clk2 (rise edge)	0.0000	0.0000
285	clock network delay (ideal)	0.0000	0.0000
286	input external delay	0.0500	0.0500 r
287	rstn (in)	0.0990	0.1490 r
288	controller_0_imem_counter_reg_4_/RN (DFFRXLTS)	0.0000	0.1490 r
289	data arrival time		0.1490
290			
291	clock clk2 (rise edge)	0.0000	0.0000
292	clock network delay (ideal)	0.0000	0.0000
293	clock reconvergence pessimism	0.0000	0.0000
294	controller_0_imem_counter_reg_4_/CK (DFFRXLTS)		0.0000 r
295	library removal time	-0.3572	-0.3572
296	data required time		-0.3572
297			
298	data required time		-0.3572
299	data arrival time		-0.1490
300			
301	slack (MET)		0.5062
302			

Fig. 12. Minimum Path Delay

392	U2287/Y (NAND4X1TS)	0.3343	26.9374 f
393	U1828/Y (OR4X2TS)	0.6809	27.6183 f
394	U1829/Y (INVX2TS)	0.1473	27.7655 r
395	U2288/Y (INVX2TS)	0.0879	27.8534 f
396	U2313/Y (AOI31X1TS)	0.2115	28.0650 r
397	U2314/Y (AOI28B2X1TS)	0.5445	28.6094 f
398	U2315/Y (INVX2TS)	0.2775	28.8869 f
399	U1456/Y (INVX2TS)	0.1026	28.9895 r
400	U1501/Y (AND3X1TS)	0.4088	29.3983 f
401	U1503/Y (INVX2TS)	0.1560	29.5543 f
402	U2363/Y (OA22X1TS)	0.4796	30.0339 f
403	U2369/Y (AOI31X1TS)	0.3350	30.3688 r
404	U2383/Y (AOI31X1TS)	0.1940	30.5628 f
405	U2385/Y (AOI211X1TS)	0.1267	30.6896 r
406	datapath_0_alu_0_fp_out_reg_9/D (DFFQX1TS)	0.0000	30.6896 r
407	data arrival time		30.6896
408			
409	clock clk2 (rise edge)	100.0000	100.0000
410	clock network delay (ideal)	0.0000	100.0000
411	clock reconvergence pessimism	0.0000	100.0000
412	datapath_0_alu_0_fp_out_reg_9/CK (DFFQX1TS)		100.0000 r
413	library setup time	-0.3261	99.6739
414	data required time		99.6739
415			
416	data required time		99.6739
417	data arrival time		-30.6896
418			
419	slack (MET)		68.9843

Fig. 13. Critical Path Delay

With the critical path delay of the circuit, we can determine the maximum frequency with the following:

$$F_{max} = \frac{1}{T_{min}} = \frac{1}{30.6896ns} = 32.58MHz$$

The designed frequency for the faster clock is 1MHz, therefore we can verify the circuit built supports the clock frequency needed for the computation of FIR Filter.

516	*****
517	Report : Time-Based Power
518	Design : fir
519	Version: U-2022.12-SP5
520	Date : Sun Dec 15 02:19:38 2024
521	*****
522	
523	
524	
525	Attributes
526	-----
527	i - Including register clock pin internal power
528	u - User defined power group
529	
530	
531	Power Group Internal Power Switching Power Leakage Power Total Power (%) Attrs
532	-----
533	clock_network 1.749e-05 0.0000 0.0000 1.749e-05 (52.37%) i
534	register 1.110e-06 1.120e-07 4.324e-08 1.265e-06 (3.79%)
535	combinational 8.578e-06 5.974e-06 4.134e-08 1.459e-05 (43.71%)
536	sequential 7.377e-09 2.744e-10 6.075e-10 8.338e-09 (0.02%)
537	memory 0.0000 0.0000 0.0000 0.0000 (0.00%)
538	io_pad 0.0000 0.0000 0.0000 0.0000 (0.00%)
539	black_box 0.0000 3.431e-08 0.0000 3.431e-08 (0.10%)
540	
541	Net Switching Power = 6.120e-06 (18.33%)
542	Cell Internal Power = 2.718e-05 (81.41%)
543	Cell Leakage Power = 8.527e-08 (0.26%)
544	-----
545	Total Power = 3.339e-05 (100.00%)

Fig. 14. Power Consumption

C. Throughput

$$\text{Throughput} = F_{max} * 16\text{bit/s} = 523\text{Mbit/s}$$

D. Energy Efficiency

Total power consumed was obtained from PrimeTime report for FIR module as shown in figure 13.

$$\text{Total Power: } 33.39 \mu W$$

Thus the energy efficiency of the FIR Filter is:

$$\begin{aligned} \text{Energy} &= \text{Power} * \text{Critical Path Delay} \\ &= 33.39\mu W * 30.6896ns \\ &= 1.025 pJ \end{aligned}$$

E. Accuracy

The accuracy of the FIR Filter is measured by Normalized Root Mean Squared Error(NRMSE), which is defined as the following:

$$NRMSE = \frac{RMSE}{y_{max} - y_{min}}$$

Where Root Mean Squared Error (RMSE) is defined as the following:

$$\sqrt{\frac{\sum_{i=0}^n (y_i - y_i^*)^2}{n}}$$

Given the fact that all of the output matched the expected output generated by MATLAB, we would expect NRMSE to be zero.

VIII. CONCLUSION

To conclude, this FIR Filter design involves generating theoretical data from MATLAB, building and testing each block component (ALU, CMEM, IMEM, FIFO), wiring them together in datapath, building controller unit that controls the entire circuit, and putting everything together into FIR module and verify its functionality with data generated in MATLAB.

The project was in general successful since we have achieved the functionality required for each building blocks and also reached perfection in calculation accuracy. However, it is worth pointing out that there might be potential improvement that can reduce the area and power of the circuit into a more efficient state.